
Trabajo Práctico N° 7

TEMA: Tipos de datos derivados estructuras. Estructuras anidadas. Arreglo de estructuras

CONSIGNA DE TRABAJO



- Trabaje de forma ordenada en cada ejercicio.
- Identifique en cada caso entre la información que se brinda los datos que conforman la estructura principal, el tipo de cada uno y la presencia de estructuras que se puedan anidar.
- En los puntos 1 y 2 trabaje declarando un dato/variable del tipo struct principal.
- En el punto 3, trabaje considerando un arreglo del tipo struct principal.
- Al trabajar con módulos (funciones/procedimientos), use pasaje de parámetros por valor del tipo struct.

1. Calentando Motores

Implementar en C un programa para el registro de una película. Considerar que tiene la siguiente información:

- **Código**
- **Título**
- **Director**:
  - Nombre
  - Apellido
  - Edad
- **Fecha de estreno**:
  - Día
  - Mes
  - Año
- **Precio base**
- **Género**: Puede ser uno de los siguientes valores: Acción, Comedia, Drama, Terror, Ciencia ficción, Romance.



Recuerde que al crear una estructura en C se está definiendo un **nuevo tipo de dato compuesto** por lo tanto puede declarar variables de este tipo**.**

Requisitos de Implementación

- Declare como estructura principal a **`Pelicula`** y como estructuras anidadas **`Director`** y **Fecha**.
- El programa debe contar con los siguientes módulos para el **tratamiento y prueba de una película:**

1. `Pelicula cargarDatos();`

   Permite ingresar la **información completa de una película**. Debe controlar:

   - Que el género ingresado sea válido.

- Que el año de estreno sea igual o menor a 2025. De no ser así, informar "Fecha inválida" y solicitar nuevamente.

2. `void mostrarDatos(Pelicula p);`

   Muestra toda la **información de una película**.

3. `float calcularCosto(Pelicula p);`

   Calcula el **costo final de una película** considerando lo siguiente:
   - Si el año de estreno es 2025 y el género es acción o ciencia ficción, se incrementa el precio base en un 50%.
   - Si el año de estreno es 2024 y el género es romance o terror, se aumenta un 25%
   - En cualquier otro caso, se cobra el precio base.

4. `int VerificarPorGenero(Pelicula p, int generoSeleccionado);`

   Crear y probar una función que:
   - Reciba como parámetros una variable de tipo película y el género a buscar.
   - Retorne 1 si la película es del género indicado.

- *Se sugiere mostrar y seleccionar el género de un arreglo fijo de cadenas.* Para esto puede realizar la siguiente declaración de forma global:

```
char genero[6][20] = {
      "Accion",
      "Comedia",
      "Drama",
      "Terror",
      "Ciencia ficcion",
      "Romance"
};
```

  El usuario deberá elegir el género de la película ingresando un número que se corresponda con uno de los declarados en el arreglo**.** Por ejemplo, `genero[1] = "Comedia"`**.**

2. Centro Cultural "Arte en Movimiento"

El Centro Cultural "Arte en Movimiento" ofrece distintos talleres artísticos durante todo el año (música, teatro, pintura, danza, etc.).

Para mantener un registro ordenado, se pide que realice un programa en lenguaje C que permita gestionar la información de un taller.

Un taller tiene un cupo máximo de 15 alumnos y debe guardar la siguiente información:

- **Nombre del taller**
- **Datos del profesor/a: Apellido, Nombre y DNI**
- **Fecha de inicio del curso**
- **Fecha de finalización del curso**
- **Costo de inscripción**
- **Lista de DNI de los alumnos inscriptos**

Su tarea consiste en:

1. Definir las estructuras para almacenar los datos de un taller, profesor y fecha.
    - La estructura **Fecha** debe contener: día, mes y año.
    - La estructura **Persona** debe contener: apellido, nombre y DNI.
    - La estructura **Taller** debe contener todos los datos generales y un arreglo para la lista de DNI.
2. Validar las fechas al momento de cargarlas:
    - Controlar que el número de días no supere el máximo posible según el mes.
    - Aceptar solo años desde el actual hasta el 2026.
3. Implementar los siguientes módulos:
    - cargarTaller() → Carga todos los datos de un taller.
    - mostrarTaller() → Muestra la información completa, incluyendo los DNI de los alumnos inscriptos.
    - actualizarCosto() → Permite incrementar el costo de inscripción según un porcentaje ingresado por teclado.
    - fechaValida() → Verifica si una fecha es válida (no contemplar años bisiestos).

 Para agilizar la carga de datos puede hacer uso de la función rand para la carga de datos numéricos aleatorios, por ejemplo:

```
40000000 + rand() % 20000000; // entre 40.000.000 y 59.999.999
```

Datos de prueba

- Taller: Pintura al óleo
- Profesor: López, Mariana – DNI 25488321
- Inicio: 12/03/2025
- Fin: 20/07/2025
- Costo: $15.000
- 3 alumnos
- Lista DNIs → 45234543, 47854798, 46547845



Arte Extremo (Bonus Track)

Modifique el módulo de carga del programa para simular la gestión de aulas dentro del centro cultural. El programa deberá permitir asignar aulas aleatoriamente a un taller, asegurando que no se repitan.

Requisitos:

1. Durante la carga de un taller, además de los datos ya solicitados (nombre, profesor, fechas, costo y alumnos), el programa deberá pedir la cantidad de aulas que se utilizarán.
2. A partir de esa cantidad, deberá generar números aleatorios en el rango de 100 a 400, representando los códigos de aula.
    - Controle que no se repita ningún número dentro del mismo taller.
    - Si el número generado ya existe, deberá volver a generar otro hasta obtener uno distinto.
    - El primer dígito del número de aula indica el piso del edificio, y los dos últimos dígitos indican el número de aula dentro de ese piso. Ejemplo: el número 214 corresponde al segundo piso, aula 14.
3. Implemente un módulo adicional que cuente y muestre cuántas aulas por piso están asignadas a ese taller.

Datos de Prueba:

- Taller: Teatro para principiantes - Profesor: Gómez, Lucía – DNI 40123567 - Fecha de inicio: 10/05/2025 - Fecha de fin: 30/11/2025 - Costo: $25.000 - 3 alumnos - Lista DNI → 45234543, 47854798, 46547845 - Cantidad de aulas utilizadas: 3	**Aulas asignadas:** 214, 221, 305 **Resultado del módulo de conteo:** Piso 2 → 2 aulas Piso 3 → 1 aula

3. Hotel manager

Desarrollar un programa en C para gestionar la **información de los huéspedes de un hotel** y sus reservas. Cada huésped puede tener **a lo sumo una sola reserva vigente**, la cual puede ser creada, visualizada o cancelada.

Cada huésped tiene los siguientes datos:

- Apellido
- Nombre
- DNI
- Fecha de nacimiento (día, mes, año)
- Teléfono de contacto
- Datos de cuenta:
    - Usuario (nombre de usuario del sistema)
    - Contraseña

La reserva contiene la siguiente información:

- Código de reserva
- Fecha de check-in (día, mes, año)
- Fecha de check-out (día, mes, año)
- Tipo de habitación (Simple, Doble, Suite)
- Monto total

Requisitos de implementación:

- Definir una estructura principal que contenga toda la información del huésped, junto a una variable que indique si hay una reserva vigente (reservaVigente vale 0 o 1). Cada huésped inicia con el dato reservaVigente en 0 (no tiene reserva).
- Crear al menos tres estructuras anidadas: una para la fecha, otra para la cuenta del usuario y una para la reserva.
- El programa debe ofrecer primero un **pre-menú interactivo** para seleccionar un huésped. Use:

  ```
  void listarHuespedes(Huesped arreglo[], int cantHuespedes);
  ```

- Una vez que el usuario selecciona un huésped, se presenta un **menú principal** con las siguientes opciones:

Opciones del menú por huésped:

1. **Agregar reserva:**
   Permite ingresar una nueva reserva para el huésped actual/seleccionado. Debe validarse:
   i. Que las fechas sean válidas y que la fecha de check-out sea posterior a la de check-in.
   ii. Que no haya una reserva vigente, de ser así debe informar y pedir la cancelación.
2. **Cancelar reserva:**
   Deja sin efecto la vigencia de una reserva ingresando su código. Para modificar la vigencia controle previamente que no se encuentre cancelada. Si ya está cancelada informe mediante un mensaje de texto.
3. **Ver reserva:**
   Muestra la reserva del huésped con sus respectivos datos, si existe.
4. **Cambiar contraseña:**
   Solicita una nueva contraseña y la actualiza en la cuenta del huésped.
5. **Ver información personal:**
   Muestra los datos básicos del huésped y sus credenciales.
6. **Volver al menú principal:**
   Permite regresar al pre-menú para seleccionar otro huésped.

Recomendaciones generales

- Trabajar cada opción del menú principal como una **función**. Usar los siguientes prototipos:

```
Huesped agregarReserva(Huesped h);
Huesped cancelarReserva(Huesped h);
void verReserva(Huesped h);
Huesped cambiarContrasenia(Huesped h);
void mostrarInformacion(Huesped h);
```

- Controlar posibles errores: fechas inválidas (puede reutilizar la función del punto 2), código de reserva no encontrado, etc.

Ejemplo de cómo inicializar datos de prueba

```
struct Huesped huespedes[3] = {
    {"Perez", "Laura", 12345678, {12, 3, 1985}, "1122334455", {"lauraP", "clave123"}, {0}, 0},
    {"Ramirez", "Carlos", 87654321, {7, 7, 1990}, "1198765432", {"carlosR", "pass456"}, {0}, 0},
    {"Martinez", "Sofia", 45678912, {20, 12, 1975}, "1100112233", {"sofiaM", "s3gura"}, {0}, 0}};
```